

Applying Design Patterns in a Microservices-Based Digital Wallet Using Spring Boot and FastAPI



Julian David Pulido Carreño¹

¹Engineer Faculty

541 (errors:3) Words

Student ID:20231020064

14th February, 2025

Abstract

The hypothesis/question and its context, scope and significance are briefly and precisely defined. The main conclusions are brief, precise and clear, noting methods used and key results. The language is clear, precise and easy to understand with no irrelevant information.

1 Introduction

Financial transactions play a fundamental role in modern economies. Traditional banking systems often introduce inefficiencies such as high transaction fees, extended processing times, and accessibility limitations. Digital financial solutions have emerged to address these challenges, offering users greater control, flexibility, and security in managing their financial resources. This paper explores the development of a digital wallet system that ensures real-time financial management using a microservices-based architecture.

Previous research highlights the significance of software design patterns in developing robust and scalable systems [gamma1994design]. Best programming practices, including those outlined in [martin2008clean], emphasize the need for modularity and maintainability to prevent common software development pitfalls. Our approach integrates design patterns and best practices to create an efficient and scalable digital wallet solution that aligns with industry standards and user expectations.

2 Methods

The digital wallet system consists of several microservices that perform distinct functionalities. These include:

- **User Authentication Service:** Developed in Java using Spring Boot to manage user authentication and authorization.
- **Transaction Service:** Implemented in Python using FastAPI, responsible for handling financial transactions securely and efficiently.
- **API Gateway:** A central communication hub that routes service requests and ensures security and scalability.

In this project, we handled two programming languages: Java for the user service and Python for transaction management. The data persistence was handled using JSON files, and the communication between services was implemented using REST. For containerization and integration, we used Docker.

The architecture follows key design patterns:

- **Command Pattern:** Encapsulates transaction operations such as deposits and withdrawals, enhancing maintainability.
- **Observer Pattern:** Implements event-driven notifications and logging, improving system monitoring and user experience.
- **Repository Pattern:** Abstracts data access layers to promote separation of concerns and maintainability.
- **API Gateway Pattern:** Centralizes service requests, enhancing security and performance management.

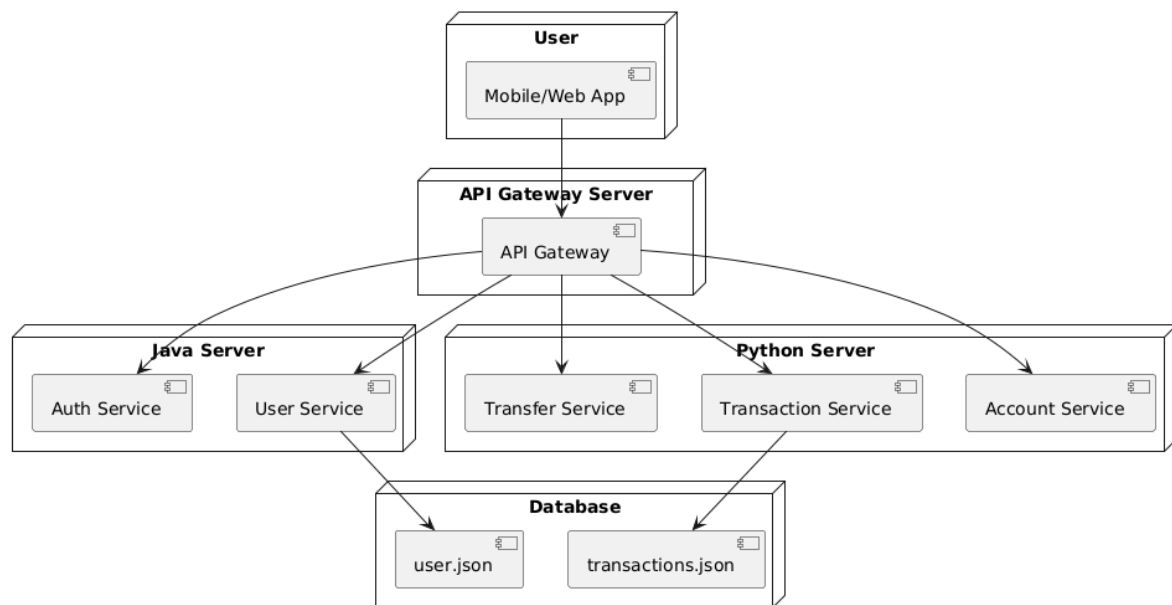


Fig. 1. Deployment Diagram of the System

3 Results

To evaluate system performance, we conducted load tests to measure response time and scalability. Results indicate that transactions are processed within an average of 2 seconds under normal load conditions, demonstrating the system's efficiency. However, under extreme loads, API Gateway congestion was observed, necessitating future optimizations.

The digital wallet project consists of two backends: one dedicated to user management and another focused on transaction handling, ensuring separation of concerns. The API Gateway facilitates communication by directing requests to the appropriate microservices. An example of this is the login process, where the API Gateway redirects requests to the user service for validation and authentication.

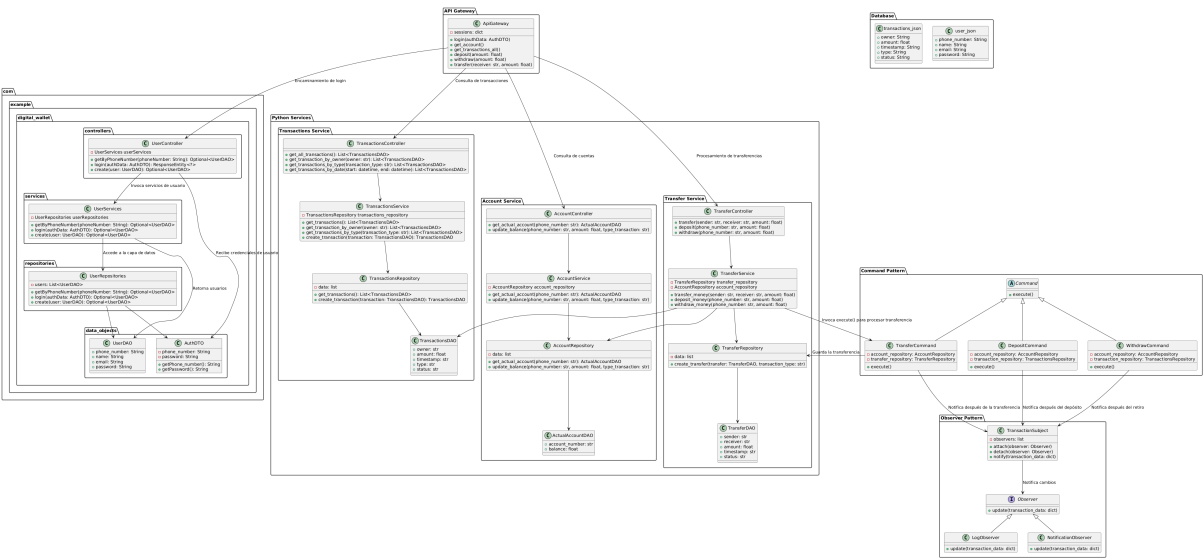


Fig. 2. Class Diagram for the Digital Wallet System

Table 1 presents a summary of design pattern effectiveness and potential issues.

Table 1
Evaluation of Design Patterns in the Digital Wallet

Pattern	Effectiveness	Potential Issue
Command	High	None observed
Observer	High	Possible overuse
Repository	High	Tight coupling with data persistence
API Gateway	Moderate	
		Overloaded gateway risk

4 Discussion

The discussion is carefully structured to provide precise and robust evidence to support the conclusions. This incorporates evidence from the results section (precisely referenced, highlighting key features) and previous work described in the introduction. A critical comparison is made between experimental results and existing solutions, addressing potential uncertainties and exploring scalability improvements.

The implementation of design patterns is a powerful approach to solving software development problems; however, excessive use of design patterns can introduce unnecessary

complexity. Ensuring the correct balance is crucial to maintaining the system's efficiency and maintainability.

5 Conclusion

This study demonstrates how applying design patterns and SOLID principles contributes to developing a scalable and maintainable digital wallet system. The architecture successfully supports modular development, reducing complexity and improving security. However, API Gateway optimization remains a key area for future research. Additionally, integrating machine learning techniques for fraud detection could further enhance system robustness.

References

- [1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.